
BA-Sauerteig

Release 1.0

Patrick Sauerteig

07.07.2026

1	src package	1
1.1	Submodules	1
1.2	src.cost module	1
1.3	src.crossing_minimization module	2
1.4	src.dblp_parser module	4
1.5	src.experiments module	5
1.6	src.hiveplot module	6
1.7	src.ip_model module	10
1.8	src.logger_setup module	13
1.9	src.main module	13
1.10	src.ordering module	14
1.11	src.partitioning module	15
1.12	src.renderer module	15
1.13	Module contents	16
	Python-Modulindex	17

1.1 Submodules

1.2 src.cost module

`src.cost.cost_function_whole`(*ordering*, *node_grps*, *edges*)

Die Kostenfunktion wird in Schritt zwei der Pipeline (Sec. 3.2) verwendet, um die kostengünstigste Achsenordnung zu ermitteln. Kostengünstig bedeutet hier, dass möglichst viele Achsenpaare mit möglichst geringem Span einfließen sollen. Diese Funktion wird für Vergleichs- und Testzwecke weiterhin bereitgestellt. :param ordering: Ordnung der Achsen (phi) :type ordering: list[int] :param node_grps: key: Achse, value: Knoten auf der Achse :type node_grps: dict[int, list[int, str]] :param edges: Kantenliste des Graphen :type edges: list[tuple[int | str, int | str]]

Rückgabe

Gesamtkosten der Achsenordnung

Rückgabetyt

int

`src.cost.edges_between_axes`(*node_grps*, *edges*, *a1*, *a2*)

Ermittelt wieviele Kanten zwischen den gegebenen Achsen bestehen.

Parameter

- **node_grps** (dict[int, list[int, str]]) – key: subset, value: list
- **edges** (list[tuple[int | str, int | str]]) – Kantenliste aus Tupeln
- **a1** (int) – Achse 1
- **a2** (int) – Achse 2

Rückgabe

Anzahl der Kanten

Rückgabetyt

int

`src.cost.node_or_axes_span`(*n1*, *n2*, *k*)

Berechnet den Span zweier Knoten oder Achsen.

Parameter

- **n1** (int) – Achse/Knoten 1
- **n2** (int) – Achse/Knoten 2
- **k** (int) – Gesamtzahl der Achsen

Rückgabe

errechneter Spann

Rückgabotyp

int

1.3 src.crossing_minimization module

`src.crossing_minimization.barycenter_crossmin_pipeline`(*layout, logger, threshold=inf, paper_like=True*)

Führt die Kreuzungsminimierungs-Pipeline mit der Barycenterheuristik aus (3a/3b).

Die Pipeline umfasst im `paper_like = false`: 1. anfängliches Layout expandieren 2. Entfernen isolierter Knoten auf den Achsen. 3. Einfügen von Dummy-Knoten für lange Kanten (`Span > 1`) und sortieren der inter axis Knoten (`Span = 0`). 4. Initialisierung des Sweeps und Updates der Datenstrukturen. 5. Mehrfache CW/CCW-Sweeps für reale und anschließend virtuelle Knoten, die die Barycenterpositionen ermitteln und die Achsen entsprechend sortieren. 6. Herstellen der Achsenordnung und letztes Update der Datenstrukturen um die Visualisierung starten zu können.

Die Pipeline umfasst im nicht `paper_like = true`: 1. Entfernen isolierter Knoten auf den Achsen. 2. Einfügen von Dummy-Knoten für lange Kanten (`Span > 1`) und sortieren der inter axis Knoten (`Span = 0`). 3. Initialisierung des Sweeps und Updates der Datenstrukturen. 4. Mehrfache CW/CCW-Sweeps für reale und anschließend virtuelle Knoten, die die Barycenterpositionen ermitteln und die Achsen entsprechend sortieren. 5. Barycenter 3b 6. Herstellen der Achsenordnung und letztes Update der Datenstrukturen um die Visualisierung starten zu können.

Parameter

- **layout** (HivePlotLayout) – Das zu optimierende HivePlotLayout.
- **threshold** (float) – Optionaler Abbruchschwellwert für die Anzahl der Sweep-Durchläufe.
- **layout_expanded** (bool) – dient der Unterscheidung, ob in der Pipeline mit expandierten Achsen gerechnet wird oder nicht, Default = False (nicht expandierter Fall)
- **paper_like** (bool) – dient der Unterscheidung zwischen identischer und unterschiedlicher Knotenordnung

Rückgabotyp

None

`src.crossing_minimization.barycenter_heuristic`(*layout, neighborhood_map, node_axis_map, threshold=inf, real=True, layout_expanded=False, use_fixed_positions=False*)

Führt wiederholte Barycenter-Sweeps im und gegen den Uhrzeigersinn über alle Achsen aus. Abhängig vom Parameter `real` werden entweder die realen Knoten (`node_groups`) oder die Dummy-Knoten (`node_groups_dummies`) pro Achse entsprechend der Barycenterposition ihrer Nachbarn umsortiert. Der Sweep endet, wenn keine Änderung mehr auftritt oder der `threshold` an Iterationen erreicht ist. Es kann zu Oszillationen im Sweep kommen, wenn z.B. wenn zwei Knoten die gleiche Barycenter Position haben oder in den Sweeps einfach nur ihre Positionen hin- und hertauschen. Dies führt dazu, dass die Schleife nie terminiert. In `state_set` werden dementsprechend alle erreichten Zustände gehasht und beim Wiederkehren eines zuvor errechneten Zustands kann die Schleife vor einem neuen Durchgang abbrechen.

Parameter

- **layout** (HivePlotLayout) – Aktuelles HivePlotLayout mit Achsenzuordnung und Knotenlisten.

- **neighborhood_map** (dict[int | str, list[int | str]]) – Map von Knoten (real und/oder Dummy, initialisierungsabhängig) auf deren Nachbarn.
- **node_axis_map** (dict[int | str, int]) – Map von Knoten auf Achsen in phi.
- **threshold** – Maximale Anzahl Sweep-Durchläufe, bevor abgebrochen wird.
- **real** (bool) – True, um reale Knoten zu sortieren; False, um Dummy-Knoten zu sortieren.
- **layout_expanded** (bool) – dient der Unterscheidung, ob in der Pipeline mit expandierten Achsen gerechnet wird oder nicht, Default = False (nicht expandierter Fall)
- **use_fixed_positions** (bool) – wird für 3b benötigt, damit fixierte positionen verwendet werden

Rückgabotyp

None

```
src.crossing_minimization.calculate_barycenter_position(layout, neighbor_group,
                                                         node_axis_map,
                                                         layout_expanded=False)
```

Berechnet die Barycenterposition eines Knotens über seine ermittelten Nachbarn. Siehe `HivePlotLayout.get_proper_neighbors()`.

Parameter

- **layout** (HivePlotLayout) – das zu berechnende HivePlotLayout
- **neighbor_group** (list[int]) – Liste der Nachbarknoten
- **layout_expanded** (bool) – dient der Unterscheidung, ob in der Pipeline mit expandierten Achsen gerechnet wird oder nicht, Default = False (nicht expandierter Fall)

Rückgabe

Barycenterposition

Rückgabotyp

float

```
src.crossing_minimization.edge_node_cleanup(layout)
```

Nachbereitung der Pipeline, dient als Absicherung, dass Kanten und intra Knoten im Layout korrekt gesetzt sind.

```
src.crossing_minimization.finish_structured_axis_orders(layout, isolated_node_groups,
                                                         layout_expanded=False)
```

Aktualisiert die node_groups am Ende der Pipeline und fügt isolierte Knoten wieder ein.

Rückgabotyp

None

```
src.crossing_minimization.parse_dummy_name(name)
```

Zerlegt den Namen eines Dummyknotens in seine Bestandteile: Startknoten, Endknoten und Sequenznummer. (siehe `make_dummy_name` Funktion für Namensschema)

Parameter

name (str) – Der zu zerlegende Dummyknoten

Rückgabe

Tupel aus int-Werten: (Startknoten, Endknoten, Sequenznummer)

Rückgabotyp

tuple[int, int, int]

```
src.crossing_minimization.remove_isolated_nodes(graph, node_groups)
```

Die Funktion entfernt alle isolierten Knoten aus der persistenten node_group des HivePlotLayouts.

Parameter

- **graph** (Graph) – Der zugrundeliegende Graph aus dem HivePlotLayout.
- **node_groups** (dict[int, list[int]]) – Die persistente Knotenlisten der Achsen.

Rückgabe

Ein dict mit den isolierten Knoten pro Achse, key: Achse, value: Listen mit isolierten Knoten.

Rückgabotyp

dict[int, list[int]]

`src.crossing_minimization.subdivide_long_edges(layout, node_position_map, node_axis_map, neighborhood_map)`

Funktion dient dem Einfügen von Dummyknoten für lange Kanten ($\text{span} > 1$) auf den Achsen zwischen Start- und Endknoten. Schema: `d_[Startknoten]_[Endknoten]_[Sequenznummer]`: z.B. `d_5_10_2` ist der zweite Dummyknoten auf der zerlegten langen Kante von 5 nach 10. Die lange Kante kann dann folgendermaßen beschrieben werden: Startknoten - `d_5_10_1` - `d_5_10_2` - ... - `d_5_10_(span-1)` - Endknoten. Außerdem werden die inter axis Knoten ($\text{Span} = 0$) ermittelt und gefiltert, um später die Achsenreihenfolge sauber zu ermitteln und die Pipelineschritte zu Initialisieren. Gefiltert bedeutet hier: 1. nur intra axis die nicht mixed sind (vgl. nested: is_mixed, werden in 3b behandelt) 2. mixed (inter + intra, werden nur in 3a nicht aber in 3b berücksichtigt) Reine intra-axis Knoten werden aus den regulären `node_groups` entfernt und ihre Kanten vorübergehend aus dem Graphen gelöscht. Am Ende der Pipeline werden alle Informationen wieder konsistent zusammengesetzt.

Parameter

- **layout** (HivePlotLayout) – Das HivePlotLayout-Objekt, das die Informationen über den Graphen, die Achsen und die Knoten enthält.
- **node_position_map** (dict[int | str, int]) – Knoten-ID: Achsenindex in phi.
- **node_axis_map** (dict[int | str, int]) – Knoten-ID: Achse
- **(neighborhood_map (neighborhood_map)** – dict[int | str, list[int | str]]): Knoten-ID: Nachbarknoten ($:= \text{proper} = \text{Span} = 1$)

Rückgabotyp

None

Rückgabe

None. Die Funktion arbeitet in-place auf dem übergebenen Layout

1.4 src.dblp_parser module

`class src.dblp_parser.LocalDTDResolver(dtd_path)`

Bases: Resolver

Da es beim Parse trotz schneller Hardware zu Performanzproblemen kam mussten Einstellungen feingranular festgelegt werden. Unter anderem in dieser Klasse gebündelt die dtd-Datei zur Übersetzung von Umlauten und Sonderzeichen. Sie wird, falls vorhanden, von lokal bevorzugt.

resolve(self, system_url, public_id, context)

Override this method to resolve an external source by `system_url` and `public_id`. The third argument is an opaque context object.

Return the result of one of the `resolve_*`() methods.

`src.dblp_parser.build_node_identity_maps(nodes)`

Funktion dient der Erstellung der Maps für die gleichnamigen Felder im Hiveplotlayout. Es werden KnotenIDs als Integer auf die Namen der Forschenden als Strings und vice versa gemappt. Dadurch wird es einerseits ermöglicht den Eingabegraphen in ein pipelinefreundliches Format zu übersetzen und gleichzeitig aus diesem am Ende die Label für den Renderer abzuleiten.

Rückgabotyp

tuple[dict[int, str], dict[str, int]]

`src.dblp_parser.parse_gd_coauthor_networks(xml_path, cache_path=None)`

Parst dblp und gibt Co-Autoren-Graphen pro Jahr zurück. Ausgabe erfolgt als pickle, um die Daten wiederverwendbar zu halten, ohne den Parse wieder durchzuführen. Lädt aus Cache falls vorhanden.

Rückgabotyp
dict[int, Graph]

`src.dblp_parser.scan_booktitles(xml_path, dtd_path)`

Vorverarbeitung der dblp-xml Datei um nach booktitles zu filtern, da die xml-Datei neben Konferenzbänden auch andere wissenschaftliche Erzeugnisse wie Dissertationen oder Buchpublikationen enthält, die nichts mit der GD-Konferenz zu tun haben.

Rückgabotyp
set[str]

1.5 src.experiments module

`class src.experiments.DataCollector(path=None)`

Bases: object

Zentrale Datenstruktur für den Datensatz der Experimente.

`data_keys(data_set=None)`

Rückgabotyp
list[str]

`data_sets: dict[str, dict[str, list]]`

`delete(data_set, x_val=None)`

Rückgabotyp
None

`get_data_set(data_set)`

`merge()`

`missing()`

Überprüft die Datensätze auf fehlende Einträge. Falls welche gefunden werden gibt es eine Liste aus Tupeln zurück, wobei das Tupel (Datensatz, x, fehlende Datenpunktbezeichner) ist. Falls x schon fehlt wird dies kenntlich gemacht.

Rückgabe
Fehlende Einträge

Rückgabotyp
list[tuple[str, str | int | float, list[str]]]

`save()`

Rückgabotyp
None

`update(data_set, x_val=None, **kwargs)`

Rückgabotyp
None

`validate(data_set)`

Ermittelt ob die Einträge des Datensatzes gleich viele Datenpunkte enthalten.

Parameter
`data_set (str)` – betrachteter Datensatz

Rückgabe

True = wenn Einträge gleiche Länge, sonst False. falls Datensatz fehlt Debug-Ausgabe

Rückgabetyt

bool | None

1.6 src.hiveplot module

```
class src.hiveplot.HivePlotLayout(graph, num_axes, axis_order, node_groups,  
                                node_groups_expanded=<factory>, edges_expanded=<factory>,  
                                node_groups_dummies=<factory>,  
                                dummy_edge_segments=<factory>, long_edges=<factory>,  
                                intra_axis_nodes=<factory>, intra_axis_edges=<factory>,  
                                id_to_name=<factory>, name_to_id=<factory>,  
                                crossings_expanded=None, mixed_nodes_by_axis=<factory>,  
                                strict_intra_nodes_by_axis=<factory>,  
                                fixed_inter_axis_delta=None, fixed_positions_by_axis=None)
```

Bases: object

Zentrale Datenstruktur für ein berechnetes Hive-Plot-Layout.

graph

ein Simple Graph den man nach der Verarbeitung der geparsten DBLP Daten erhält: V = Autoren, E = Coautorenschaft zu

Type

nx.Graph

num_axes

k = Anzahl Achsen

Type

int

axis_order

phi

Type

list[int]

node_groups

alpha: siehe node_groups Funktion in ordering.py

Type

dict[int, list[int]]

node_groups_dummies

Achsen mit Dummyknoten

Type

dict[int, list[int]]

dummy_edge_segments

key: Kantentupel (u,v) value: Liste von Kantensegmenten die u und v über Dummyknoten verbinden

Type

dict[tuple[int,int], list[tuple[int, int]]]

node_order

pi_i pro Achse gebündelt

Type

dict[int, list[int]]

crossings

Kreuzungszahl Standardmodell

Type

Optional[int]

crossings_extended

Kreuzungszahl erweitertes Modell

Type

Optional[int]

axis_order: list[int]

classify_nodes_for_3b()

Klassifiziert die Knoten für Pipelineschritt 3b.

Rückgabetyt

None

copy()

Gibt eine Deepcopy des Hiveplotlayouts zurück.

Rückgabetyt

copy

count_crossings(layout_expanded=False)

Funktion zählt die induzierten Kanten des fertigen Hiveplotlayout.

Rückgabetyt

int

crossings_expanded: int | None = None

dummy_edge_segments: list[tuple[int | str, int | str]]

edges()

Gibt die Kantenliste des Graphen zurück. Notwendig, weil mit dem Networkx Edgeview nicht direkt gearbeitet werden kann.

Rückgabe

Liste der Kanten als Tupel (u, v)

Rückgabetyt

list[tuple]

edges_expanded: list[tuple[int | str, int | str]]

fixed_inter_axis_delta: dict | None = None

fixed_positions_by_axis: dict | None = None

freeze_barycenter_positions(layout_expanded=False)

Legt die fixierten Positionen für den Pipelineschritt 3b der Barycenterpipeline fest. Wird für alle mixed Knoten ermittelt.

freeze_inter_axis_delta()

Legt die delta Map für den Pipelineschritt 3b der ILP-Pipeline fest. Wird für alle mixed Knoten ermittelt.

Rückgabetyt

None

fuse_edges_with_edge_dummies()

Erzeugt eine Liste die alle kurzen Kanten und Dummykanten vereinigt zurückgibt.

Rückgabe

Vereinigung aus Kanten und Dummykanten

Rückgabetyt

list[tuple[int | str, int | str]]

fuse_node_groups_with_dummies(layout_expanded=False)

Erzeugt ein dict mit Achsen als Schlüssel und der vereinigten Menge aus Knoten und Dummyknoten pro Achse als Wert. Zuerst die realen dann die virtuellen Knoten. :param layout_expanded: dient der Unterscheidung, ob in der Pipeline mit expandierten Achsen gerechnet wird oder nicht, Default = False (nicht expandierter Fall) :type layout_expanded: bool

Rückgabe

Vereinigung der Knoten und Dummyknoten pro Achse.

Rückgabetyt

dict[int, list[int | str]]

get_proper_neighborhood_map(fused_edge_list, layout_expanded=False)

Die Funktion ermittelt eine Liste, die jeden Knoten (sowohl real als auch virtuell) auf eine Liste seiner Nachbarn mappt.

Parameter

- **fused_edge_list** (list[tuple[int | str, int | str]]) – list[tuple[int | str, int | str]]: Liste der realen und virtuellen Kantentupel
- **layout_expanded** (bool) – dient der Unterscheidung, ob in der Pipeline mit expandierten Achsen gerechnet wird oder nicht, Default = False (nicht expandierter Fall)

Rückgabe

list[int | str]: KnotenID: Nachbarliste

Rückgabetyt

dict[slice(int | str, list[int | str], None)]

graph: Graph

id_to_name: dict[int, str]

intra_axis_edges: list[tuple[int, int]]

intra_axis_nodes: dict[int, list[int]]

long_edges: set[tuple[int, int]]

mixed_nodes_by_axis: dict[int, list[int | str]]

name_to_id: dict[str, int]

node_groups: dict[int, list[int | str]]

node_groups_dummies: dict[int, list[str]]

node_groups_expanded: dict[int, list[int | str]]

num_axes: int

post_processing_expansion(node_axis_map, dummy_copy=None)

Die Funktion dient dem post-processing des Eingabegraphen nach der Pipeline, sodass bei diesem die Achsen expandiert werden. Dazu wird die Achse i (> 0) zu den Achsen mit KnotenIDs i und $-i$ expandiert. Die intra-axis Kanten werden symmetrisch zwischen den Achsenkopien gezeichnet. Folgende

Schritte werden durchgeführt: 1. Initialisieren der Dummysegmente, diese werden in der Funktion multiert 2. Initialisieren der Map intra_expandables mit Key AchsenID einer Kante und ihrer intra-axis Kanten (Achse wird nur aufgenommen, wenn es intra-axis Kanten gibt) 3. Initialisieren der Map inter_expandables mit allen Kanten die genau einen Start oder Enknoten auf der expandierten Achse haben 4. Initialisieren der node_groups_expanded mit Keys ursprüngliche AchsenIDs und expandierte AchsenIDs und ihre in node_groups enthaltenen Knotenlisten (expandierte Achse -i bekommt alle Knoten von i zugeordnet, jedoch werden die KnotenIDs negativ gesetzt, für virtuelle Knoten wird die neue Sequenznummer ermittelt und die neue Kante zwischen den virtuellen Knoten direkt in das Networkx Graphmodell hinzugefügt) 5. Update des Hiveplotlayouts mit neuer Achsenordnung und -anzahl und Update der edge_axis_map 6. Initialisieren einer axis_position_map mit Key AchsenID und Value Position der Achse in der neuen Achsenordnung 7. Behandlung der intra-axis Kanten

- a. Entfernen der intra-axis Kanten aus dem Hiveplotlayout
- b. Erstellen der neuen intra-axis Kanten zwischen den expandierten Achsen (Kante (u,v) auf Achse i wird symmetrisch gespiegelt zu den Kanten (-u, v) und (u, -v))
- c. Einpflegen der neuen intra-axis Kanten in die Networkx Graphenstruktur des Hiveplotlayouts

8. Behandlung der inter-axis Kanten

- a. Entfernen der inter-axis Kanten aus dem Hiveplotlayout und den Dummykantensegmenten
- b. Erstellen der neuen inter-axis Kanten, Fallbehandlung:

I: Sonderfall: sowohl u als auch v auf expandierten achsen

i.) v muss aktualisiert werden

<1> v ist real und muss negiert werden <2> v ist dummy und muss aktualisiert werden

ii.) u muss aktualisiert werden

<1> u ist real und muss negiert werden <2> u ist dummy und muss aktualisiert werden

II: u auf expandierter achse, v auf nicht expandierter achse

i.) u ist real und muss negiert werden ii.) u ist dummy und muss aktualisiert werden

III: v auf expandierter achse, u auf nicht expandierter achse

i.) v ist real und muss negiert werden ii.) v ist dummy und muss aktualisiert werden

- c. Einpflegen der neuen inter-axis Kanten in die Networkx Graphenstruktur des Hiveplotlayouts

9. alle neuen Knoten in hiveplot.graph.nodes speichern und subset zuordnen

Parameter

- **node_axis_map** (dict[int | str, int]) – Knoten-ID: Achsen-ID
- **dummy_copy** (list[tuple[int | str, int | str]]) – eine Aktuelle Kopie der Dummysegmente, Default ist None

Rückgabetyt

None

pre_processing_expansion(node_axis_map)

Die Funktion dient der Umsetzung unterschiedlicher Knotenordnungen auf expandierten Achsen. Die Funktion ist ausschließlich zum pre-processing des Eingabegraphen bevor etwaige Berechnungen der Pipeline durchgeführt werden gedacht. Dazu wird die Achse i zu den Achsen mit KnotenIDs i und -i expandiert. Die intra-axis Kanten werden symmetrisch zwischen den Achsenkopien gezeichnet. Folgende Schritte werden durchgeführt: 1. Initialisieren der Map intra_expandables mit Key AchsenID einer Kante und ihrer intra-axis Kanten (Achse wird nur aufgenommen, wenn es intra-axis Kanten gibt) 2. Initialisieren der Map inter_expandables mit allen Kanten die genau einen Start oder Enknoten auf der

expandierten Achse haben 3. Initialisieren der `node_groups_expanded` mit Keys ursprüngliche AchsenIDs und expandierte AchsenIDs und ihre in `node_groups` enthaltenen Knotenlisten (expandierte Achse -i bekommt alle Knoten von i zugeordnet, jedoch werden die KnotenIDs negativ gesetzt) 4. Update des `Hiveplotlayouts` mit neuer Achsenordnung und -anzahl 5. Initialisieren einer `axis_position_map` mit Key AchsenID und Value ist Position der Achse in der neuen Achsenordnung 6. Behandlung der intra-axis Kanten

- a. Entfernen der intra-axis Kanten aus dem `Hiveplotlayout`
- b. Erstellen der neuen intra-axis Kanten zwischen den expandierten Achsen (Kante (u,v) auf Achse i wird symmetrisch gespiegelt zu den Kanten (-u, v) und (u, -v))
- c. Einpflegen der neuen intra-axis Kanten in die `Networkx` Graphenstruktur des `Hiveplotlayouts`

7. Behandlung der inter-axis Kanten

- a. Entfernen der inter-axis Kanten aus dem `Hiveplotlayout`
 - b. **Erstellen der neuen inter-axis Kanten:**
I: Ziel- und Startknoten der Kante auf expandierter Achse: betrachte Kanten i/j mit Knoten u/v wenn i vor j in der Achsenordnung, Kante (u, v) zu (-u, v) andernfalls zu (u, -v) [i -> -i -> j -> -j] II: Ziel- oder Startknoten der Kante auf expandierter Achse: betrachte Kanten i/j mit Knoten u/v (i expandiert), falls i vor j in der Achsenordnung (u,v) zu (-u, v), andernfalls Kante übernehmen [j -> i -> -i]
 - c. Einpflegen der neuen inter-axis Kanten in die `Networkx` Graphenstruktur des `Hiveplotlayouts`
8. Einpflegen und Zuordnung zu Subsets der neuen Knoten in der `Networkx` Graphenstruktur des `Hiveplotlayouts`

Parameter

node_axis_map (`dict[int | str, int]`) – Knoten-ID: Achsen-ID

Rückgabotyp

None

strict_intra_nodes_by_axis: `dict[int, list[int | str]]`

updated_nodes(*fused_node_groups*)

Gibt die Knotenliste des Graphen zurück. Notwendig, weil die Knoten sich ändern und je nach Pipeschritt die Knoten geupdated werden müssen.

Parameter

fused_node_groups (`dict[int, list[int | str]]`) – AchsenID: Knotenliste (real und virtuell)

Rückgabe

Liste der realen und virtuellen Knoten

Rückgabotyp

`list[int]`

1.7 src.ip_model module

src.ip_model.delta_mapping(*fused_groups*)

Dient der Initialisierung der Ordnungsvariablen ($\delta^i_{u,v}$) für das 1L2S-ILP. Erzeugt aus der Vereinigung von virtuellen und realen Knoten das Mapping.

Parameter

layout (`HivePlotLayout`) – das zugrundeliegende `Hiveplotlayout`

Rückgabe

key: (u, v, alpha(u,v)), value: None

Rückgabety

dict[tuple[int | str, ...], 1]

src.ip_model.get_delta_value(delta, fixed_delta, u, v, axis)

Die Funktion gibt das korrekte delta für fixierte oder nicht fixierte Ordnung zurück. Notwendig für 3b.

Rückgabety

tuple[int | str, int | str, int]

src.ip_model.ilp_name(*parts)

src.ip_model.induced_crossings(prob, delta_static, fixed_delta, sorted_neighbors, pi_fix, pi_var_id, pi_var)

Ermittelt die Summanden der Zielfunktion für induzierte Kreuzungen. Für jedes Knotenpaar der variablen Achse werden alle Nachbargaare auf der festen Achse betrachtet. Kreuzungen werden durch die Linearisierung der Produkte der entsprechenden Ordnungsvariablen modelliert. Wir verwenden eine allgemeine Linearisierung, da die Funktion get_delta_value sowohl feste Werte als auch PuLP-Variablen zurückgeben kann. Die Zielfunktion bleibt äquivalent zu Nöllenburg und Wallinger, sie wird dadurch nur robust für den Solver.

Parameter

- **prob** (LpProblem) – das zugrunde liegende ILP-Modell
- **delta_static** (dict[tuple[int | str, int | str, int], object]) – bereits bekannte Ordnungsvariablen
- **fixed_delta** (dict[tuple[int | str, int | str, int], int]) – fixierte delta-Werte
- **sorted_neighbors** (dict[tuple[int, int | str], list[int | str]]) – Nachbarschaftsmap
- **pi_fix** (int) – ID feste Achse
- **pi_var_id** (int) – ID variable Achse
- **pi_var** (list[int | str]) – Knoten der variablen Achse

Rückgabe

lineare Zielfunktion zur Minimierung induzierter Kreuzungen

Rückgabety

object

src.ip_model.ip_model_pipeline(layout, logger, threshold=10, paper_like=True)

Die Funktion führt die gesamte Pipeline zur Kreuzungsminimierung durch, inklusive der Berechnung des 1L2S-ILP. Alle Schritte werden in-place am HivePlotLayout durchgeführt. Ablauf paper_like = false: 1. anfängliches Layout expandieren 2. isolierte Knoten entfernen 3. Segmentierung 4. 1L2S-ILP berechnen 5. Herstellen der Achsenordnung und letztes Update der Datenstrukturen um die Visualisierung starten zu können.

Ablauf paper_like = true: 1. isolierte Knoten entfernen 2. Segmentierung 3. 1L2S-ILP berechnen 3a 4. 1L2S-ILP berechnen 3b 5. Herstellen der Achsenordnung und letztes Update der Datenstrukturen um die Visualisierung starten zu können.

Parameter

- **layout** (HivePlotLayout) – das zugrundeliegende Hiveplotlayout
- **threshold** (int) – Anzahl der Sweeps bis zum Abbruch, Defaultwert ist 10.
- **layout_expanded** (bool) – dient der Unterscheidung, ob in der Pipeline mit expandierten Achsen gerechnet wird oder nicht, Default = False (nicht expandierter Fall)
- **paper_like** (bool) – dient der Unterscheidung zwischen identischer und unterschiedlicher Knotenordnung

Rückgabotyp

None

`src.ip_model.linearize_product(prob, a, b, name)`

Die Funktion kapselt die Linearisierung der quadratischen Terme in der Zielfunktion. Legt sowohl Variablen an als auch Nebenbedingungen fest.

Rückgabotyp

LpVariable

`src.ip_model.onelayer_twosided_optimization(layout, neighborhood_map, node_axis_map,
threshold=10, layout_expanded=False,
paper_like=False)`

Enthält die gesamte Logik, um das One Layer Two Sided Integer Linear Program (1L2S ILP) zu definieren und zu berechnen. Die Funktion verändert das HiveplotLayout in-place. Die Pipeline besteht aus Sweeps. Ein Sweep umfasst eine clockwise und eine counterclockwise Berechnung über jede Achse des Hiveplots. Folgende Schritte werden in der Funktion durchgeführt: 1. Initialisierung der Ordnungsvariablen ($\delta^i_{u,v}$) durch `delta_mapping()` 2. Initialisierung der Sweepsschleife, Abbruch bei Erreichen eines Thresholds 3. Danach schließt sich ein Sweep an erst in clockwise dann counterclockwise Richtung, beide haben das gleiche Schema:

- a. Erstellen einer Problemistanz und einer Kopie für die aktuelle Achse
 - b. **Initialisierung der Ordnungsvariablen für die aktuelle Achse (virtuelle und reale Knotenordnung werden in einem Schritt berechnet, aber die Achsenordnung $\text{real} < \text{virtuell}$ muss berücksichtigt werden, da Gaps nicht explizit behandelt werden)**
I: Ordnungsvariablen getrennt für reale und virtuelle Achsenabschnitte II: Ordnungsvariablen für $\text{real} < \text{virtuell} = \text{konstant 1}$ III: Ordnungsvariablen für $\text{virtuell} < \text{real} = \text{konstant 0}$
 - c. Erstellen der Zielfunktion speziell für die variable Achse
 - d. **Erstellen der Nebenbedingungen für:**
I: Antisymmetrie (im Paper implizit behandelt) II: Transitivität
 - e. Lösen des ILP
 - f. Synchronisieren der Lösung mit δ
4. Aktualisieren des Layouts durch Übersetzung von δ zurück in die Achsenordnung, da die variable Achse auf einem geupdateten Stand ermittelt werden muss, wird dies am Ende eines Sweeps durchgeführt

Parameter

- **layout** (HivePlotLayout) – das zugrundeliegende HivePlotLayout
- **neighborhood_map** (dict[str | int, list[int | str]]) – KnotenID: Liste von proper Nachbarn
- **node_axis_map** (dict[str | int, int]) – KnotenID: Achse
- **threshold** (int) – Anzahl der Sweeps (=1x cw+ 1x ccw), Defaultwert ist 10
- **layout_expanded** (bool) – dient der Unterscheidung, ob in der Pipeline mit expandierten Achsen gerechnet wird oder nicht, Default = False (nicht expandierter Fall)

Rückgabotyp

None

`src.ip_model.onelayer_twosided_optimization_3b(layout, neighborhood_map, node_axis_map,
threshold=10, layout_expanded=False,
paper_like=False)`

Die Funktion arbeitet analog zu `onelayer_twosided_optimization()`, berücksichtigt allerdings die zuvor fixierte globale Ordnung. Die Funktion wird explizit für Pipelineschritt 3b benötigt.

Parameter

- **layout** (HivePlotLayout) – das zugrundeliegende HivePlotLayout
- **neighborhood_map** (dict[str | int, list[int | str]]) – KnotenID: Liste von proper Nachbarn

- **node_axis_map** (dict[str | int, int]) – KnotenID: Achse
- **threshold** (int) – Anzahl der Sweeps (=1x cw+ 1x ccw), Defaultwert ist 10
- **layout_expanded** (bool) – dient der Unterscheidung, ob in der Pipeline mit expandierten Achsen gerechnet wird oder nicht, Default = False (nicht expandierter Fall)

Rückgabety

None

1.8 src.logger_setup module

`src.logger_setup.log(logger, msg)`

`src.logger_setup.setup_logger(log_dir, per_session=True, file_name=None)`

Konfiguriert den Logger und initialisiert ihn. Kann für Batchläufe eine Datei schreiben und für Einzeldurchläufe mehrere.

Parameter

- **log_dir** (Path) – Ablagepfad
- **per_session** (bool) – Batch- oder Einzelauswahl. Defaults to True.
- **file_name** (str | None) – Dateiname. Defaults to None.

Rückgabe

der angelegte Log

Rückgabety

Logger

1.9 src.main module

`src.main.init_graph(original, name_to_id)`

`src.main.init_hiveplot(year, own_pkl, partitions, logger)`

Rückgabety

HivePlotLayout

`src.main.init_logger(per_session)`

`src.main.init_original(year, own_pkl=None)`

Lädt einen Graphen aus dem Cache oder einer eigenen Pickle-Datei.

Parameter

- **year** (int) – gewünschte GD-Jahr
- **own_pkl** (str | None) – Dateiname einer .pkl-Datei in LOAD_DIR. Muss ein nx.Graph-Objekt enthalten.

Rückgabe

das Graph-Objekt

Rückgabety

Graph

`src.main.main()`

`src.main.pipeline(year, run, logger, output_name, variant, paper_like, partitions=8, threshold=5, save=False, debug=False, batch=False, own_pkl=None, gui=False)`

`src.main.save_pkl(hiveplot, name, save, logger)`

Rückgabotyp

None

`src.main.save_rendered_hiveplot(svg_path, year, logger)`

Rückgabotyp

None

`src.main.step_ordering(hiveplot, logger)`

Rückgabotyp

None

1.10 src.ordering module

`src.ordering.brute_force_ordering(ordering, node_grps, edges)`

Berechnet die optimale Knotenordnung nach der Kostenfunktion über alle Permutationen von k. Diese Funktion wird für Vergleichs- und Testzwecke weiterhin bereitgestellt. :param ordering: aktuelle Achsenordnung (phi) :type ordering: list[int] :param node_grps: key: Achse, value: Knoten :type node_grps: dict :param edges: Knotenliste :type edges: list

Rückgabe

Optimierte Achsenordnung

Rückgabotyp

tuple[int]

`src.ordering.ip_ordering(layout)`

Rückgabotyp

list[int]

`src.ordering.node_to_axis(node, node_grps, position=False)`

Gibt an auf welcher Achse sich ein Knoten befindet. Funktion wird nur zum Initialisieren verwendet. Für Iterationen in der Pipeline ist sie nicht performant. Dafür werden Mappings benutzt (z.B. siehe ordering.node_to_axis_maps)

Parameter

- **node** (*int*) – Knoten ID
- **node_grps** (*dict*) – key: Achse, value: Knoten
- **position** (*bool*) – Rückgabe der Achse, true Rückgabe der Position der Achse in der Achsenordnung (phi). False ist default.

Verursacht

ValueError – Der Knoten kann keiner Achse zugeordnet werden.

Rückgabe

Achsen ID

Rückgabotyp

int

`src.ordering.node_to_axis_maps(layout, fused_node_groups)`

Die Funktion berechnet zwei Maps um für einen gegebenen Knoten sehr schnell 1. alpha(Knoten) und 2. phi[alpha(Knoten)] zu bestimmen.

Parameter

- **layout** (HivePlotLayout) – Das Hiveplotlayout
- **fused_node_groups** (dict[int, list[int | str]]) – Achse: Knotenliste (virtuell und real)

Rückgabe

Knoten: Achse & Knoten: Achsenposition in Phi

Rückgabotyp

tuple[dict[int | str, int], dict[int | str, int]]

`src.ordering.reordered_node_groups(node_grps, new_order)`

Schnelles Umordnen nach Optimierung der Achsenordnung.

Rückgabotyp

dict[int, list[int]]

1.11 src.partitioning module

`src.partitioning.cluset_newman_moore_communities(graph, threshold=0)`

Berechnet die Communities eines Graphen per Cluset-Newman-Moore Algorithmus und wandelt die frozenset Liste in ein dict um. Falls ein Threshold übergeben wird, so wird die Anzahl der Communities auf den Threshold gestaucht. So wird sichergestellt, dass man eine Anzahl Communities (Achsen) erhält, die noch einen übersichtlichen Hiveplot erzeugen lassen.

Parameter

- **graph** (*nx.Graph*) – Der Eingabegraph für den die Communities berechnet werden sollen.
- **threshold** (int) – maximale Anzahl Communities, bei Default (=0) wird die natürliche Anzahl von Communities ermittelt und zurückgegeben

Rückgabe

Ein Dictionary mit den Communities als Werte und ihren IDs als Schlüssel.

Rückgabotyp

dict[int, list[int]]

`src.partitioning.louvain_community_detection(graph)`

Berechnet die Communities eines Graphen per Louvain Community Detection und wandelt die frozenset Liste in ein dict um. Diese Funktion wird für Vergleichs- und Testzwecke weiterhin bereitgestellt. :type graph: nx.Graph :param graph: Der Eingabegraph, für den die Communities berechnet werden sollen. :type graph: nx.Graph

Rückgabe

Ein Dictionary mit den Communities als Werte und ihren IDs als Schlüssel.

Rückgabotyp

dict[int, list[int]]

1.12 src.renderer module

`src.renderer.draw_basis(layout, node_groups, edges, degree=False, id_to_label_map=None, unordered=False, intra_edges=None, axes_labels=True)`

Bündelt die Darstellung von Kanten, Knoten, Labels und Achsen.

Rückgabotyp

tuple[list[str], list[str], list[str], list[str]]

`src.renderer.hiveplot_renderer(name, layout, debug_dir, degree=True, layout_expanded=False, intra=False, node_labels=True, axes_labels=True, unordered=False, paper_like=False)`

Pipeline die das Rendern des fertig berechneten Hiveplotlayouts in eine Scalable Vector Graphics (SVG) realisiert.

Rückgabotyp

Path

`src.renderer.render_svg(filename, width, height, elements)`

Setzt den Header der .svg Datei und hängt die Graphenelemente an und setzt die Schlussklausel. Datei wird neu erstellt bzw. überschrieben, falls sie bereits existiert.

Rückgabetyp

None

`src.renderer.translate_polar_to_carthesian(radius, angle, center_x=500.0, center_y=500.0)`

Umrechnung der Polarkoordinate in ein karthesisches Tupel.

Rückgabetyp

tuple[float, float]

1.13 Module contents

Top-level package for project source files.

S

- `src`, 16
 - `src.cost`, 1
 - `src.crossing_minimization`, 2
 - `src.dblp_parser`, 4
 - `src.experiments`, 5
 - `src.hiveplot`, 6
 - `src.ip_model`, 10
 - `src.logger_setup`, 13
 - `src.main`, 13
 - `src.ordering`, 14
 - `src.partitioning`, 15
 - `src.renderer`, 15

A

`axis_order` (Attribut von `src.hiveplot.HivePlotLayout`), 6, 7

B

`barycenter_crossmin_pipeline()` (im Modul `src.crossing_minimization`), 2

`barycenter_heuristic()` (im Modul `src.crossing_minimization`), 2

`brute_force_ordering()` (im Modul `src.ordering`), 14

`build_node_identity_maps()` (im Modul `src.dblp_parser`), 4

C

`calculate_barycenter_position()` (im Modul `src.crossing_minimization`), 3

`classify_nodes_for_3b()` (Methode von `src.hiveplot.HivePlotLayout`), 7

`clauset_newman_moore_communities()` (im Modul `src.partitioning`), 15

`copy()` (Methode von `src.hiveplot.HivePlotLayout`), 7

`cost_function_whole()` (im Modul `src.cost`), 1

`count_crossings()` (Methode von `src.hiveplot.HivePlotLayout`), 7

`crossings` (Attribut von `src.hiveplot.HivePlotLayout`), 6

`crossings_expanded` (Attribut von `src.hiveplot.HivePlotLayout`), 7

`crossings_extended` (Attribut von `src.hiveplot.HivePlotLayout`), 7

D

`data_keys()` (Methode von `src.experiments.DataCollector`), 5

`data_sets` (Attribut von `src.experiments.DataCollector`), 5

`DataCollector` (Klasse in `src.experiments`), 5

`delete()` (Methode von `src.experiments.DataCollector`), 5

`delta_mapping()` (im Modul `src.ip_model`), 10

`draw_basis()` (im Modul `src.renderer`), 15

`dummy_edge_segments` (Attribut von `src.hiveplot.HivePlotLayout`), 7

E

`edge_node_cleanup()` (im Modul `src.crossing_minimization`), 3

`edges()` (Methode von `src.hiveplot.HivePlotLayout`), 7

`edges_between_axes()` (im Modul `src.cost`), 1

`edges_expanded` (Attribut von `src.hiveplot.HivePlotLayout`), 7

F

`finish_structured_axis_orders()` (im Modul `src.crossing_minimization`), 3

`fixed_inter_axis_delta` (Attribut von `src.hiveplot.HivePlotLayout`), 7

`fixed_positions_by_axis` (Attribut von `src.hiveplot.HivePlotLayout`), 7

`freeze_barycenter_positions()` (Methode von `src.hiveplot.HivePlotLayout`), 7

`freeze_inter_axis_delta()` (Methode von `src.hiveplot.HivePlotLayout`), 7

`fuse_edges_with_edge_dummies()` (Methode von `src.hiveplot.HivePlotLayout`), 7

`fuse_node_groups_with_dummies()` (Methode von `src.hiveplot.HivePlotLayout`), 8

G

`get_data_set()` (Methode von `src.experiments.DataCollector`), 5

`get_delta_value()` (im Modul `src.ip_model`), 11

`get_proper_neighborhood_map()` (Methode von `src.hiveplot.HivePlotLayout`), 8

`graph` (Attribut von `src.hiveplot.HivePlotLayout`), 6, 8

H

`hiveplot_renderer()` (im Modul `src.renderer`), 15

`HivePlotLayout` (Klasse in `src.hiveplot`), 6

I

`id_to_name` (Attribut von `src.hiveplot.HivePlotLayout`), 8

`ilp_name()` (im Modul `src.ip_model`), 11

- induced_crossings() (im Modul *src.ip_model*), 11
 init_graph() (im Modul *src.main*), 13
 init_hiveplot() (im Modul *src.main*), 13
 init_logger() (im Modul *src.main*), 13
 init_original() (im Modul *src.main*), 13
 intra_axis_edges (Attribut von *src.hiveplot.HivePlotLayout*), 8
 intra_axis_nodes (Attribut von *src.hiveplot.HivePlotLayout*), 8
 ip_model_pipeline() (im Modul *src.ip_model*), 11
 ip_ordering() (im Modul *src.ordering*), 14
- ## L
- linearize_product() (im Modul *src.ip_model*), 12
 LocalDTDResolver (Klasse in *src.dblp_parser*), 4
 log() (im Modul *src.logger_setup*), 13
 long_edges (Attribut von *src.hiveplot.HivePlotLayout*), 8
 louvain_community_detection() (im Modul *src.partitioning*), 15
- ## M
- main() (im Modul *src.main*), 13
 merge() (Methode von *src.experiments.DataCollector*), 5
 missing() (Methode von *src.experiments.DataCollector*), 5
 mixed_nodes_by_axis (Attribut von *src.hiveplot.HivePlotLayout*), 8
 module
 - src*, 16
 - src.cost*, 1
 - src.crossing_minimization*, 2
 - src.dblp_parser*, 4
 - src.experiments*, 5
 - src.hiveplot*, 6
 - src.ip_model*, 10
 - src.logger_setup*, 13
 - src.main*, 13
 - src.ordering*, 14
 - src.partitioning*, 15
 - src.renderer*, 15
- ## N
- name_to_id (Attribut von *src.hiveplot.HivePlotLayout*), 8
 node_groups (Attribut von *src.hiveplot.HivePlotLayout*), 6, 8
 node_groups_dummies (Attribut von *src.hiveplot.HivePlotLayout*), 6, 8
 node_groups_expanded (Attribut von *src.hiveplot.HivePlotLayout*), 8
 node_or_axes_span() (im Modul *src.cost*), 1
 node_order (Attribut von *src.hiveplot.HivePlotLayout*), 6
 node_to_axis() (im Modul *src.ordering*), 14
 node_to_axis_maps() (im Modul *src.ordering*), 14
- num_axes (Attribut von *src.hiveplot.HivePlotLayout*), 6, 8
- ## O
- onelayer_twosided_optimization() (im Modul *src.ip_model*), 12
 onelayer_twosided_optimization_3b() (im Modul *src.ip_model*), 12
- ## P
- parse_dummy_name() (im Modul *src.crossing_minimization*), 3
 parse_gd_coauthor_networks() (im Modul *src.dblp_parser*), 4
 pipeline() (im Modul *src.main*), 13
 post_processing_expansion() (Methode von *src.hiveplot.HivePlotLayout*), 8
 pre_processing_expansion() (Methode von *src.hiveplot.HivePlotLayout*), 9
- ## R
- remove_isolated_nodes() (im Modul *src.crossing_minimization*), 3
 render_svg() (im Modul *src.renderer*), 15
 reordered_node_groups() (im Modul *src.ordering*), 15
 resolve() (Methode von *src.dblp_parser.LocalDTDResolver*), 4
- ## S
- save() (Methode von *src.experiments.DataCollector*), 5
 save_pkl() (im Modul *src.main*), 13
 save_rendered_hiveplot() (im Modul *src.main*), 14
 scan_booktitles() (im Modul *src.dblp_parser*), 5
 setup_logger() (im Modul *src.logger_setup*), 13
 src
 - module, 16
 - src.cost*
 - module, 1
 - src.crossing_minimization*
 - module, 2
 - src.dblp_parser*
 - module, 4
 - src.experiments*
 - module, 5
 - src.hiveplot*
 - module, 6
 - src.ip_model*
 - module, 10
 - src.logger_setup*
 - module, 13
 - src.main*
 - module, 13
 - src.ordering*
 - module, 14
 - src.partitioning*

module, [15](#)
src.renderer
 module, [15](#)
step_ordering() (*im Modul src.main*), [14](#)
strict_intra_nodes_by_axis (Attribut von
 src.hiveplot.HivePlotLayout), [10](#)
subdivide_long_edges() (*im Modul*
 src.crossing_minimization), [4](#)

T

translate_polar_to_carthesian() (*im Modul*
 src.renderer), [16](#)

U

update() (Methode von
 src.experiments.DataCollector), [5](#)
updated_nodes() (Methode von
 src.hiveplot.HivePlotLayout), [10](#)

V

validate() (Methode von
 src.experiments.DataCollector), [5](#)